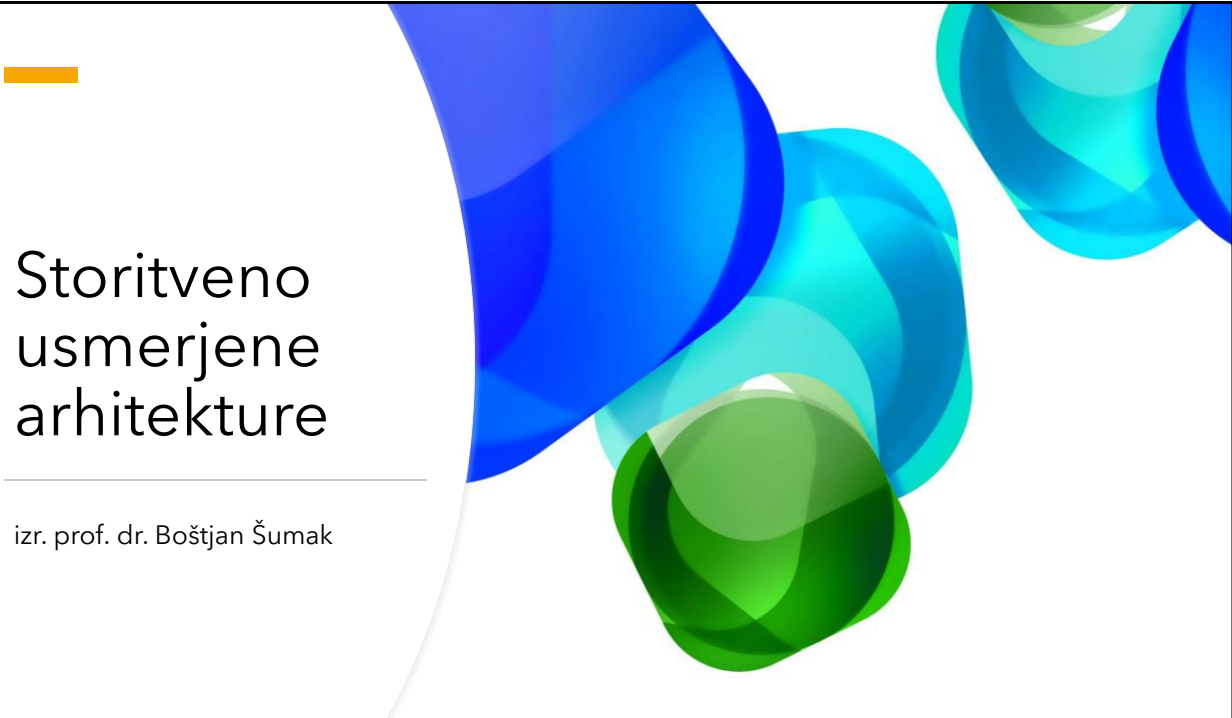
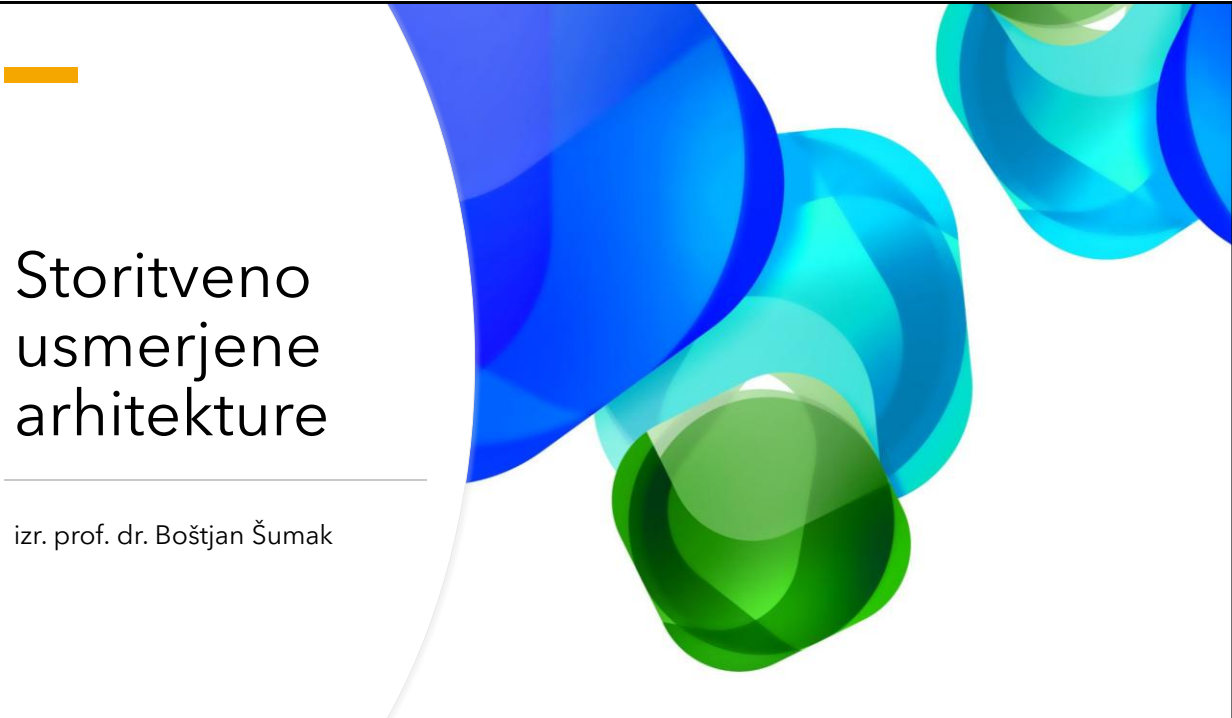




Storitveno usmerjene arhitekture

izr. prof. dr. Boštjan Šumak

1



Komunikacijski protokoli in tehnologije razvoj storitev in aplikacijskih programskih vmesnikov

2

Zakaj so komunikacijski protokoli in tehnologije za razvoj API-jev pomembni?

• Povezovanje aplikacij

- omogočajo povezovanje aplikacij, sistemov in naprav v heterogenih okoljih.
- Brez njih bi bila interoperabilnost med komponentami skoraj nemogoča.

• Temelj porazdeljenih sistemov in storitev

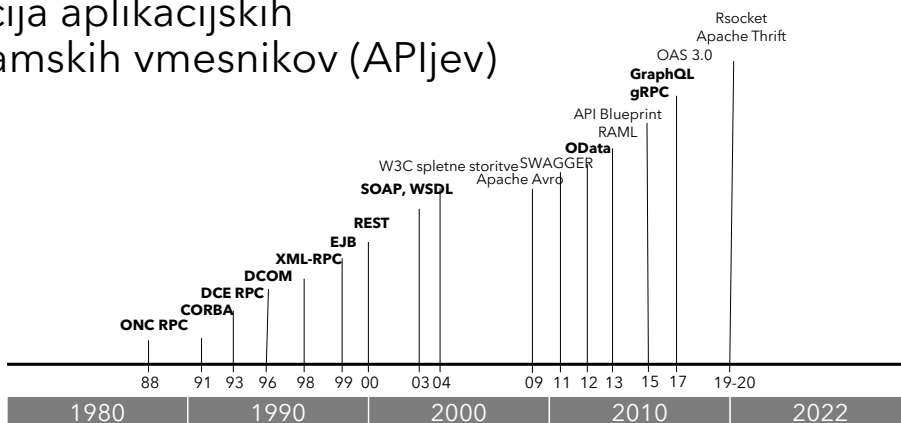
- Porazdeljeni sistemi so osnova za sodobne aplikacije (mikroservisi, IoT, oblačne rešitve).

• Gradniki sodobnih rešitev

- Aplikacijski programski vmesniki (API-ji) so način, kako aplikacije izmenjujejo podatke in zagotavljajo storitve.

3

Evolucija aplikacijskih programskih vmesnikov (APIjev)



- Koncept aplikacijskih programskih vmesnikov (APIji) je prisoten že od samega začetka porazdeljenega računalništva
- Začetke porazdeljenega programiranja računalništva zasledimo že v 1960, ko so bili ustvarjeni prvi protokoli tipa zahteva-odgovor

4

Tehnologije za oddaljeno proženje procedur

- Namen tehnologije za oddaljeno proženje procedur je **zagotavljanje večje interoperabilnosti aplikacij, ki tečejo na heterogenih platformah**
- Zasnovano na proceduralnih konceptih – razvijalci uporabljajo oddaljene procedure ali funkcijske klice
- Prve implementacije so bile na voljo v zgodnjih 1980

5

Klici oddaljenih procedur (rutin)

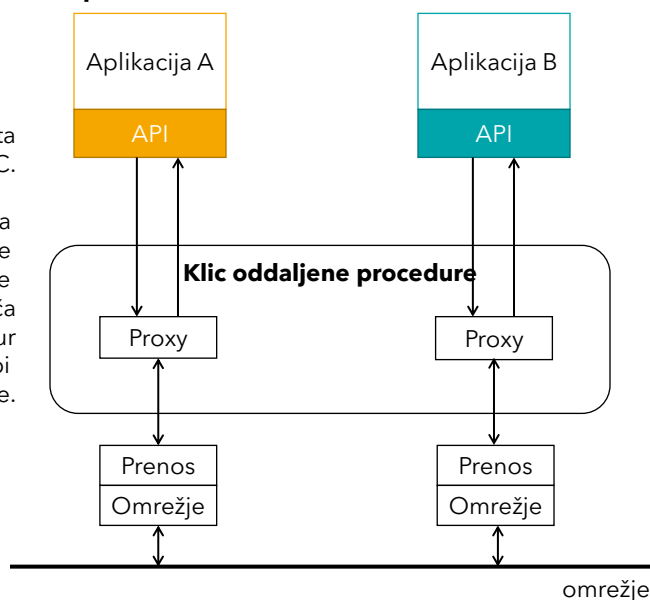
- Remote Procedure Call – RPC
- Oblika interakcije **odjemalec-strežnik**, ki se običajno izvede v obliki izmenjave sporočil preko protokola **zahteva-odgovor**
 - Nujno je, da je **odjemalec dosegljiv v času pošiljanja odgovora**
- Proženje oddaljene procedure pomeni, da
 - **računalniški program sproži izvajanje procedure (rutine) na drugem naslovu** (v splošnem na drugem računalniku čez omrežje)
 - Proženje procedure se programira **enako kot da bi klicali lokalno** proceduro
 - Programerju **ni potrebno eksplicitno navesti podrobnosti interakcije** z oddaljenim računalnikom

6

Klici oddaljenih procedur

Aplikaciji komunicirata preko sistema RPC.

Vmesna povezovalna oprema skriva vse podrobnosti glede komunikacije, kar omogoča klicanje oddaljenih procedur na enak način, kot če bi klicali lokalne procedure.



7

RPC – glavne prednosti

Abstrakcija kompleksnosti

- Programerji lahko uporabljajo oddaljene procedure na enak način kot lokalne, kar poenostavi razvoj aplikacij.

Transparentnost

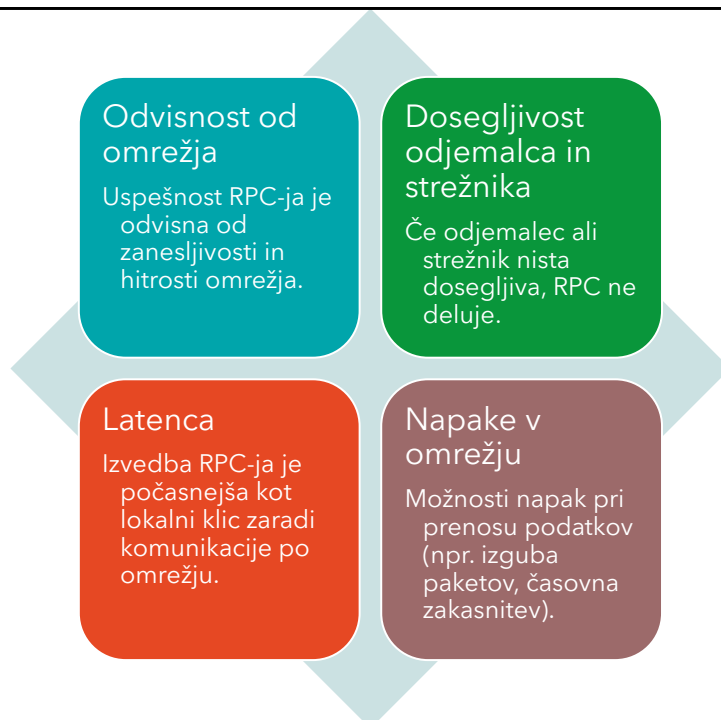
- Podrobnosti o omrežni komunikaciji, kot so protokoli, kodiranje sporočil in prenos podatkov, so skrite pred programerjem.

Ponovna uporaba

- Omogoča ponovno uporabo obstoječih funkcij ali storitev na oddaljenih računalnikih brez potrebe po njihovem prepisovanju.

8

RPC – slabosti in omejitve



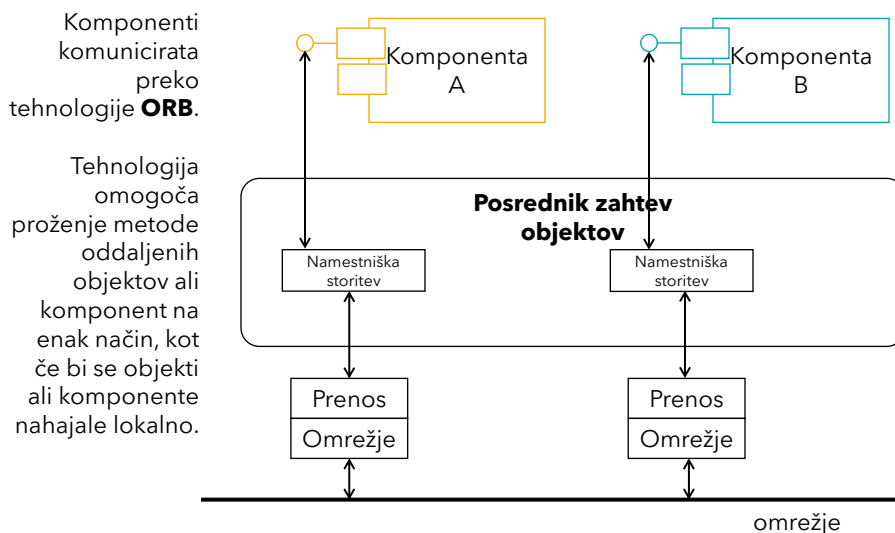
9

Posredniki zahtev objektov

- Object Request Broker (ORB)
- Tehnologija na vmesnem povezovalnem sloju, ki omogoča **upravljanje in komunikacijo med porazdeljenimi objekti/komponentami**
- Tehnologija, ki **skrije podrobnosti komunikacije** in omogoča interoperabilnost med porazdeljenimi objekti/komponentami
- **Komunikacija** med porazdeljenimi objekti/komponentami temelji na **vmesnikih** in je običajno **sinhrona**
- Tehnologija, ki omogoča neodvisnost od
 - lokacije,
 - programskega jezika,
 - protokolov,
 - operacijskih sistemov

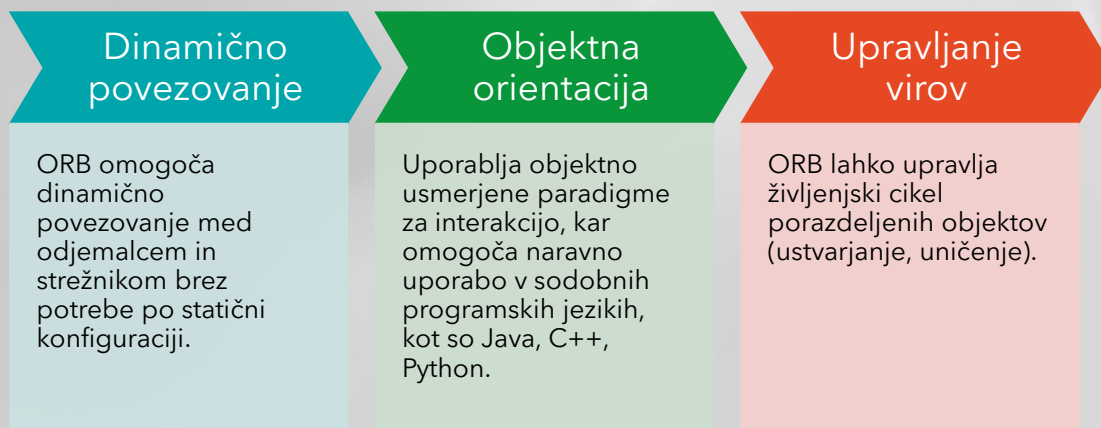
10

Posredniki zahtev objektov



11

ORB - ključne funkcionalnosti



12

ORB - glavne prednosti

Abstrakcija kompleksnosti

- Skrije podrobnosti, kot so protokoli, lokacija objektov, in transportni sloj komunikacije.

Večja interoperabilnost

- Omogoča sodelovanje med različnimi aplikacijami in platformami.

Skalabilnost

- ORB se lahko uporablja v majhnih sistemih ali velikih porazdeljenih okoljih.

13

ORB - omejitve

CORBA in podobni standardi so pogosto kompleksni za implementacijo in vzdrževanje.

Latenca

Sinhrona narava komunikacije lahko povzroča zakasnitve, še posebej v omrežjih z visoko latenco.

Kompleksnost

Odvisnost od standardov

Različne implementacije ORB morda ne zagotavljajo popolne interoperabilnosti, če ne sledijo enakim standardom.

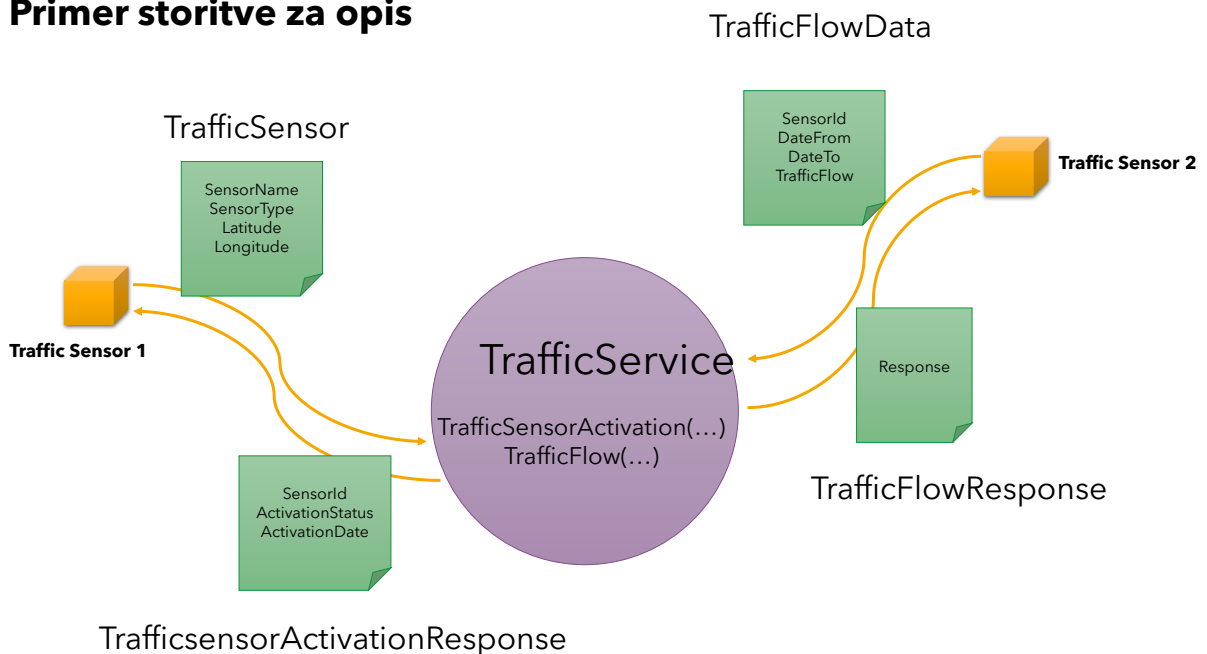
14

ONC RPC

- **Open Network Computing (ONC) Remote Procedure Call (RPC)**
- Sistem za oddaljeno proženje procedur, ki ga je razvilo podjetje Sun Microsystems v letu 1980
 - zasnovan za interoperabilnost v heterogenih omrežjih in je postal temelj številnih sistemov, vključno z NFS (Network File System).
- Jezik za definicijo vmesnika: **RPC IDL**
- Vsebina sporočila (payload): Podatki, serializirani na osnovi XDR
- Prenosni protokol: UDP ali TCP
- Prva izdaja: 1984

15

Primer storitve za opis



16

```

/* TrafficService RPC Interface Definition */

/* Določitev različice protokola */
program TRAFFIC_SERVICE_PROG {
    version TRAFFIC_SERVICE_VERS {
        /* Aktivacija senzorja za promet */
        TrafficSensorActivationResponse TrafficSensorActivation(TrafficSensorActivationRequest) = 1;
        /* Pridobitev podatkov o prometnem toku */
        TrafficFlowResponse TrafficFlow(TrafficFlowRequest) = 2;
    } = 1;
} = 0x20000001;

/* Definicije podatkovnih struktur */

/* Zahteva za aktivacijo senzorja */
struct TrafficSensorActivationRequest {
    string SensorName<128>;
    string SensorType<64>;
    string Longitude<32>;
    string Latitude<32>;
};

/* Odgovor na aktivacijo senzorja */
struct TrafficSensorActivationResponse {
    string SensorId<64>;
    string ActivationStatus<32>;
    string ActivationDate<32>;
};

/* Zahteva za podatke o prometu */
struct TrafficFlowRequest {
    string TrafficId<64>;
    string DateFrom<32>;
    string DateTo<32>;
    string TrafficFlow<128>;
};

/* Odgovor na podatke o prometu */
struct TrafficFlowResponse {
    string Response<256>;
};

```

ONC RPC IDL - primer definicije vmesnika

17

Glavni elementi ONC RPC IDL

• program

- Definira celoten RPC vmesnik.
- TRAFFIC_SERVICE_PROG je ime vmesnika.

• version

- Omogoča različne verzije istega RPC vmesnika.
- TRAFFIC_SERVICE_VERS je verzija (prva verzija ima številko 1).

• struct

- Definira podatkovne strukture za zahteve in odgovore.
- Določen je vsak atribut, skupaj z omejitvami dolžine nizov (npr. string SensorName<128>).

• Funkcije

- TrafficSensorActivation in TrafficFlow so metode, ki jih lahko kličemo prek RPC.
- Vsaka metoda ima unikatni identifikator (npr. = 1 za prvo operacijo).

18

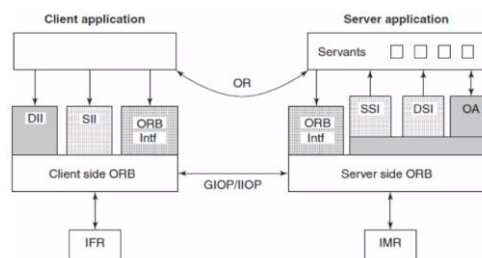
CORBA

- **Common Object Request Broker Architecture (CORBA)**
- Standard, definiran iz strani Object Management Group (OMG)
- Standard je bil oblikovan za omogočanje komunikacije sistemov, ki so nameščeni na različnih platformah
 - operacijskih sistemih, programskih jeziki in strojni opremi
- CORBA uporablja objektno-orientiran model
 - sistemi, ki uporabljajo CORBA ni nujno, da so razviti na objektno orientiranih programskih platformah

19

CORBA

- Jezik za definicijo vmesnika: **IDL - Interface Definition Language** z možnostjo preslikave v specifične programske jezike.
- Vsebina sporočila (payload): **Internet Inter-ORB Protocol (IIOP)**, binarni protokol za komunikacijo med ORB-ji.
- Prenosni protokol: Primarno **TCP/IP**, kasneje tudi HTTP za izboljšanje interoperabilnosti.
- Prva izdaja: **1991**.



Vir: <https://www.slideshare.net/PriyankaPatil41/corba-basic-and-deployment-of-corba>

20

CORBA IDL - primer definicije vmesnika

```
module TrafficService {
    // Struktura za zahtevo aktivacije senzorja
    struct TrafficSensorActivationRequest {
        string SensorName;
        string SensorType;
        string Longitude;
        string Latitude;
    };
    // Struktura za odgovor aktivacije senzorja
    struct TrafficSensorActivationResponse {
        string SensorId;
        string ActivationStatus;
        string ActivationDate;
    };
    // Struktura za zahtevo podatkov o prometu
    struct TrafficFlowRequest {
        string TrafficId;
        string DateFrom;
        string DateTo;
        string TrafficFlow;
    };
    // Struktura za odgovor na podatke o prometu
    struct TrafficFlowResponse {
        string Response;
    };
    // Definicija vmesnika za storitev
    interface TrafficServiceInterface {
        // Operacija za aktivacijo senzorja
        TrafficSensorActivationResponse TrafficSensorActivation(
            in TrafficSensorActivationRequest request
        );
        // Operacija za pridobitev podatkov o prometnem toku
        TrafficFlowResponse TrafficFlow(
            in TrafficFlowRequest request
        );
    };
};
```

21

Glavni elementi CORBA IDL

• module

- Omogoča organizacijo imen v logičnih sklopih.
- V tem primeru smo vse vključili v modul TrafficService.

• struct

- Definira kompleksne podatkovne tipe.
- Uporablja se za vhodne in izhodne podatke operacij (npr. TrafficSensorActivationRequest in TrafficSensorActivationResponse).

• interface

- Definira vmesnik, ki vključuje operacije, ki jih lahko kličejo odjemalci.
- Vsaka operacija ima vhodne (oznaka in) in izhodne parametre (oznaka out je neobvezna za rezultate, ker tu uporabljamo vrnjeno vrednost za izhod).

• Podatkovni tipi

- string: Uporablja se za nizovne podatke.
- CORBA IDL podpira tudi druge tipe, kot so short, long, float, boolean, sequence, itd.

22

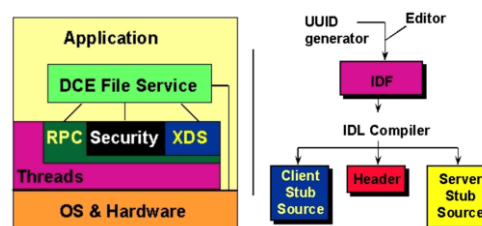
DCE RPC

- **Distributed Computing Environment (DCE) RPC**
- RPC sistem, ki ga je vpeljala neprofitna organizacija Open Software Foundation (OSF)
- V DCE RPC ustvarimo razrede odjemalca in strežnika s prevajanjem opisa oddaljenega vmesnika (datoteke z definicijo vmesnika)

23

DCE RPC

- Jezik za definicijo vmesnika: **IDL - Interface Definition Language** (datoteke za definicijo vmesnikov).
- Vsebina sporočila (payload): IDL podatkovni tipi se preslikajo v binarne tokove (osmiške podatkovne nize).
- Prenosni protokol: **TCP/IP, UDP** (SMB/SMB2 se uporabljata v nekaterih Microsoftovih implementacijah DCE RPC).
- Prva izdaja: **1993**.



Vir: <https://www.soa4u.co.uk/2019/02/a-brief-look-at-evolution-of-interface.html>

24

```

[
    uuid(12345678-1234-5678-1234-567812345678),
    version(1.0)
]
interface TrafficServiceInterface {
    typedef struct {
        [string] char* SensorName;
        [string] char* SensorType;
        [string] char* Longitude;
        [string] char* Latitude;
    } TrafficSensorActivationRequest;
    typedef struct {
        [string] char* SensorId;
        [string] char* ActivationStatus;
        [string] char* ActivationDate;
    } TrafficSensorActivationResponse;
    typedef struct {
        [string] char* TrafficId;
        [string] char* DateFrom;
        [string] char* DateTo;
        [string] char* TrafficFlow;
    } TrafficFlowRequest;
    typedef struct {
        [string] char* Response;
    } TrafficFlowResponse;
    error_status_t TrafficSensorActivation(
        [in] TrafficSensorActivationRequest* request,
        [out] TrafficSensorActivationResponse* response
    );
    error_status_t TrafficFlow(
        [in] TrafficFlowRequest* request,
        [out] TrafficFlowResponse* response
    );
}

```

DCE RPC IDL - primer definicije vmesnika

25

Glavni elementi DCE RPC IDL

- **interface**
 - Definira ime vmesnika, ki ga odjemalci in strežniki uporabljajo za komunikacijo.
 - UUID (Universally Unique Identifier) zagotavlja edinstvenost vmesnika.
- **typedef struct**
 - DCE RPC IDL uporablja C-jevsko sintakso za opis podatkovnih struktur.
 - Strukture so definirane za zahteve in odgovore posameznih operacij.
- **error_status_t**
 - Privzeta vrsta za povratne vrednosti operacij, ki omogoča povratne kode za uspeh ali napake.
- **Atributi**
 - **[in]**: Označuje vhodni parameter za operacijo.
 - **[out]**: Označuje izhodni parameter, ki ga metoda vrne.
 - **[string]**: Označuje, da je podatek niz (char pointer), ki je zaključen z NULL.
- **UUID in verzija**
 - **uuid**: Definira edinstven identifikator vmesnika, ki je potreben za DCE RPC.
 - **version**: Omogoča določanje verzij vmesnikov.

26

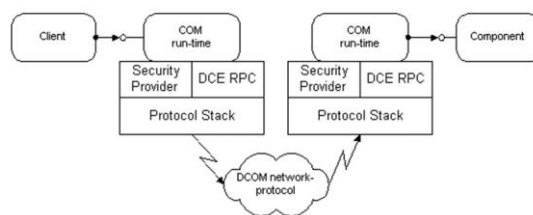
DCOM

- **Distributed Component Object Model (DCOM)**
- Osnovna tehnologija podjetja Microsoft za omogočanje komunikacije med programskimi komponentami, ki tečejo na računalnikih v omrežju
- Glavna konkurenčna tehnologija tehnologiji CORBA

27

DCOM

- Jezik za definicijo vmesnika: **Microsoft IDL** (MIDL) z razširitvijo **ACF** (Application Configuration File).
- Vsebina sporočila (payload): **Serializirani DCOM objekti** (lastniška implementacija Microsofta).
- Prenosni protokol: Primarno **TCP/IP in NetBIOS**, pozneje tudi HTTP.
- Prva izdaja: Prvi razvoj 1990, prva javna objava **1996** (Windows NT 4.0).



Vir: <https://www.soa4u.co.uk/2019/02/a-brief-look-at-evolution-of-interface.html>

28

```

import "oaidl.idl";
import "ocidl.idl";
[
    uuid(87654321-4321-8765-4321-876543218765),
    version(1.0)
]
library TrafficServiceLibrary {
    importlib("stdole32.tlb");
    // Definicija vmesnika za storitev
    [
        object,
        uuid(12345678-1234-5678-1234-567812345678),
        pointer_default(unique)
    ]
    interface ITrafficService : IUnknown {
        HRESULT TrafficSensorActivation(
            [in] BSTR SensorName,
            [in] BSTR SensorType,
            [in] BSTR Longitude,
            [in] BSTR Latitude,
            [out, retval] BSTR* ActivationResponse
        );
        HRESULT TrafficFlow(
            [in] BSTR TrafficId,
            [in] BSTR DateFrom,
            [in] BSTR DateTo,
            [in] BSTR TrafficFlowRequest,
            [out, retval] BSTR* TrafficFlowResponse
        );
    };
    [
        uuid(23456789-2345-6789-2345-678923456789),
        version(1.0)
    ]
    coclass TrafficService {
        [default] interface ITrafficService;
    };
};

```

DCOM IDL - primer definicije vmesnika

29

Glavni elementi MIDL

- **library**
 - Glavna komponenta IDL datoteke, ki združuje vse definicije.
 - V tem primeru smo vse vključili v knjižnico TrafficServiceLibrary.
- **import**
 - Uvozi standardne definicije, ki so potrebne za DCOM, kot sta oaidl.idl (OLE Automation) in ocidl.idl (OLE Control).
- **interface**
 - Definira DCOM vmesnik, ki ga odjemalci uporabljajo za interakcijo s storitvijo.
 - IUnknown: Osnovni vmesnik za vse COM/DCOM objekte.
 - Metode vmesnika:
 - HRESULT: Standardna vrnjena vrednost za uspeh ali napake.
 - [in]: Označuje vhodne parametre.
 - [out, retval]: Označuje izhodni parameter, ki se vrne kot rezultat.
- **coclass**
 - Definira komponento (razred), ki implementira vmesnik.
 - TrafficService: Implementira vmesnik ITrafficService.
- **UUID in verzije**
 - uuid: Unikaten identifikator za knjižnico, vmesnike in razrede.
 - version: Verzija knjižnice ali komponente.
- **Podatkovni tipi**
 - BSTR: OLE Automation string (unicode nizi), pogosto uporabljen v DCOM aplikacijah.

30

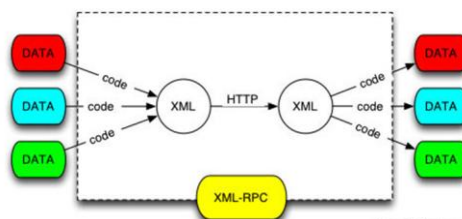
XML-RPC

- **Extensible Markup Language (XML) Remote Procedure Call (RPC)**
- Protokol, ki uporablja tehnologijo XML za zapis klicev oddaljenih procedur in HTTP za prenos zahtev čez omrežje
- V XML-RPC odjemalec izvede RPC tako, da pošlje zahtevo HTTP na strežnik, ki implementira XML-RPC, in prejme odgovor HTTP. Klic lahko vsebuje več parametrov in vrne en rezultat.
- Protokol definira osnovne podatkovne tipe, kot so številke, nizi, datumi, matrike in strukture.

31

XML-RPC

- Jezik za definicijo vmesnika: **Ni posebnega jezika za definicijo vmesnikov (vse je v XML formatu).**
- Vsebina sporočila (payload): **Serializirana XML sporočila**, ki vsebujejo zahtevo ali odgovor.
- Prenosni protokol: HTTP
- Prva izdaja: 1998



Source: JY Services

Vir: <http://1998.xmlrpc.com/>

32

```

<?xml version="1.0"?>
<service name="TrafficService" xmlns="http://example.org/xmlrpc-service">
  <methods>
    <method name="TrafficSensorActivation">
      <description>Aktivira senzor na lokaciji.</description>
      <params>
        <param name="SensorName" type="string" />
        <param name="SensorType" type="string" />
        <param name="Longitude" type="string" />
        <param name="Latitude" type="string" />
      </params>
      <returns>
        <param name="Response" type="struct">
          <field name="SensorId" type="string" />
          <field name="ActivationStatus" type="string" />
          <field name="ActivationDate" type="string" />
        </param>
      </returns>
    </method>
    <method name="TrafficFlow">
      <description>Pridobi podatke o prometu.</description>
      <params>
        <param name="TrafficId" type="string" />
        <param name="DateFrom" type="string" />
        <param name="DateTo" type="string" />
        <param name="TrafficFlow" type="string" />
      </params>
      <returns>
        <param name="Response" type="struct">
          <field name="Response" type="string" />
        </param>
      </returns>
    </method>
  </methods>
</service>

```

XML RPC - primer definicije vmesnika

33

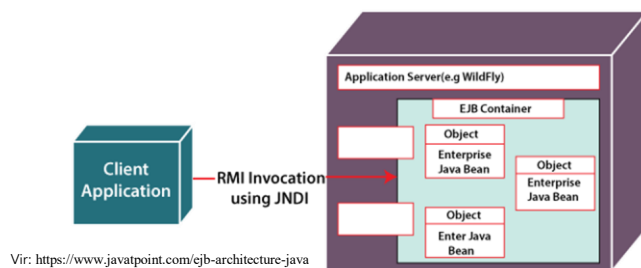
EJB

- **Enterprise Java Beans (EJB)**
- Tehnologija za razvoj strežniških programskih komponent, ki enkapsulirajo poslovno logiko aplikacije in omogočajo transakcije, varnost ter skalabilnost.
 - EJB je podmnožica Java EE (zdaj Jakarta EE) specifikacij.
- Zrna so bila zasnovana za razvoj visoko-skalabilnih in robustnih poslovnih aplikacij na Java EE aplikacijskih strežnikih
 - npr. GlassFish, JBoss, WebLogic, WildFly, ...

34

EJB

- Jezik za definicijo vmesnika: **Java Interface Definition Language (Java IDL)** definira metode, ki jih lahko proži odjemalec
- Vsebina sporočila (payload): **Sprva serializirani Java objekti** (DTO), kasneje podpora **za XML in JSON** prek HTTP in REST API-jev.
- Prenosni protokol: rimarno Java RMI, kasneje HTTP za boljšo interoperabilnost.
- Prva izdaja: 1998



35

EJB - primer definicije vmesnika

```
package com.example.traffic;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface TrafficServiceRemote extends Remote {

    /**
     * Activates a traffic sensor.
     * @param sensorName Name of the sensor.
     * @param sensorType Type of the sensor.
     * @param longitude Longitude of the sensor location.
     * @param latitude Latitude of the sensor location.
     * @return Response containing activation details.
     * @throws RemoteException if the remote invocation fails.
     */
    String trafficSensorActivation(String sensorName, String sensorType, String longitude, String latitude)
        throws RemoteException;

    /**
     * Retrieves traffic flow data.
     * @param trafficId ID of the traffic flow data request.
     * @param dateFrom Start date for the data.
     * @param dateTo End date for the data.
     * @return Response containing traffic flow details.
     * @throws RemoteException if the remote invocation fails.
     */
    String trafficFlow(String trafficId, String dateFrom, String dateTo) throws RemoteException;
}
```

36

Glavni gradniki EJB

- **Oddaljeni vmesnik** (TrafficServiceRemote):
 - Definira metode, ki so dostopne odjemalcem.
 - Deduje od java.rmi.Remote.
 - Metode lahko vržejo RemoteException.
- **EJB Implementacija** (TrafficServiceBean):
 - Označena z anotacijo @Stateless, kar pomeni, da je EJB brez stanja.
 - Implementira oddaljeni vmesnik in vsebuje poslovno logiko.
- **JNDI:**
 - Java Naming and Directory Interface omogoča odjemalcem iskanje EJB komponent.
- **EJB container:**
 - Upravljalno okolje, ki zagotavlja storitve, kot so transakcije, varnost in obdelava virov.

37

REST

- **Representational State Transfer (REST)**
- Arhitekturni stil za gradnjo porazdeljenih sistemov
- Temelji na množici principov.
- Glavni cilj je zagotavljanje enostavnosti, zmogljivosti in skalabilnosti.

38

REST

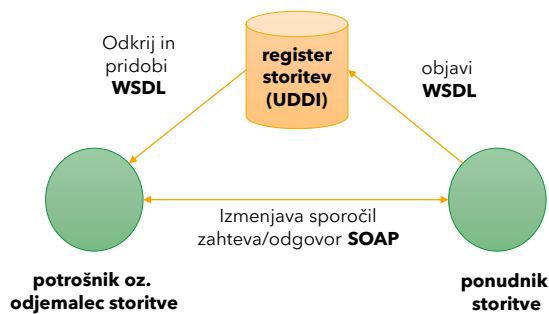
- REST ne predpisuje jezika za definicijo vmesnika,
 - so pa sčasoma nastali jeziki, kot so Swagger/OpenAPI, RAML, in API Blueprint.
- Vsebina sporočila (payload):
 - REST ne predpisuje specifičnega formata; najpogosteje se uporablja JSON, sledijo XML in drugi formati.
- Prenosni protokol:
 - REST običajno uporablja HTTP, vendar lahko deluje na kateremkoli protokolu, kot so WebSocket ali FTP.
- Prva izdaja:
 - Koncept REST je bil prvič predstavljen leta **2000** v disertaciji Royja Fieldinga.

Vir: <https://www.linkedin.com/pulse/understanding-rest-architecture-gabriel-gitonga/>

39

SOAP/WSDL

- Jezik za definicijo vmesnika: WSDL **Web Services Description Language (WSDL)**
- Vsebina sporočila (payload): XML na osnovi protokola SOAP
- Prenosni protokol: HTTP
- Prva izdaja: **2003**



40

OData

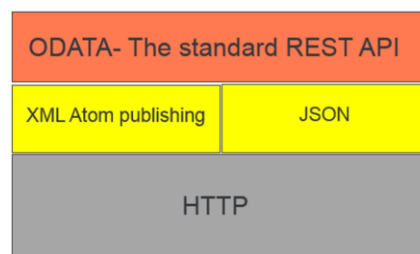
• Open Data Protocol (OData)

- Protokol za standardiziran dostop do podatkov, zasnovan na REST principih, ki omogoča interakcijo z podatkovnimi viri preko HTTP in URI.
- Razvit iz strani podjetja Microsoft
 - pozneje pa je postal odprti standard organizacije OASIS in je odobren tudi kot standard ISO/IEC.
- Cilj: **Poenostavitev in standardizacija osnovnih operacij (CRUD) na podatkovnih virih v obliki REST.**

41

OData

- Jezik za definicijo vmesnika: **CSDL** (Common Schema Definition Language), sprva v XML formatu, kasneje pa tudi v JSON.
- Vsebina sporočila (payload): **Primarno JSON**, z možnostjo podpore za XML.
- Prenosni protokol: **HTTP**.
- Prva izdaja: Prvič predstavljen leta **2007** kot del Microsoftove platforme, formalno standardiziran leta 2012.



Vir: <https://www.soa4u.co.uk/2019/02/a-brief-look-at-evolution-of-interface.html>

42

```

<?xml version="1.0" encoding="UTF-8"?>
<edmx:Edmx Version="4.0" xmlns:edmx="http://docs.oasis-open.org/odata/ns/edmx">
  <edmx:DataServices>
    <Schema Namespace="TrafficService" xmlns="http://docs.oasis-open.org/odata/ns/edm">

      <!-- EntityType za senzor -->
      <EntityType Name="TrafficSensor">
        <Key>
          <PropertyRef Name="SensorId" />
        </Key>
        <Property Name="SensorId" Type="Edm.String" Nullable="false" />
        <Property Name="SensorName" Type="Edm.String" />
        <Property Name="SensorType" Type="Edm.String" />
        <Property Name="Longitude" Type="Edm.String" />
        <Property Name="Latitude" Type="Edm.String" />
        <Property Name="ActivationStatus" Type="Edm.String" />
        <Property Name="ActivationDate" Type="Edm.String" />
      </EntityType>

      <!-- EntityType za podatke o prometu -->
      <EntityType Name="TrafficFlow">
        <Key>
          <PropertyRef Name="TrafficId" />
        </Key>
        <Property Name="TrafficId" Type="Edm.String" Nullable="false" />
        <Property Name="DateFrom" Type="Edm.String" />
        <Property Name="DateTo" Type="Edm.String" />
        <Property Name="TrafficFlowData" Type="Edm.String" />
      </EntityType>

      <EntityContainer Name="TrafficServiceContainer">
        <EntitySet Name="TrafficSensors" EntityType="TrafficService.TrafficSensor" />
        <EntitySet Name="TrafficFlows" EntityType="TrafficService.TrafficFlow" />
      </EntityContainer>

    </Schema>
  </edmx:DataServices>
</edmx:Edmx>

```

Odata (XML) - primer definicije vmesnika

43

Elementi OData CSDL

- **<edmx:Edmx>**
 - Koreninski element, ki vsebuje vse informacije o modelu.
 - Vsebuje **<edmx:DataServices>**, kjer je definiran model podatkov.
- **<Schema>**
 - Definira entitete, komplekse tipe in operacije.
 - Atribut Namespace določa ime prostora imen (v tem primeru TrafficService).
- **<EntityType>**
 - Predstavlja podatkovno entiteto (npr. TrafficSensor, TrafficFlow).
 - **<Key>**: Primarni ključ entitete.
 - **<Property>**: Lastnosti entitete, s tipi podatkov, ki sledijo standardu OData (npr. Edm.String, Edm.Int32).
- **<EntityContainer>**
 - Definira entitetske sete (kolekcije entitet), ki so dostopne prek OData API-ja.
 - **<EntitySet>**: Predstavlja zbirko entitet (npr. TrafficSensors).

44

GraphQL

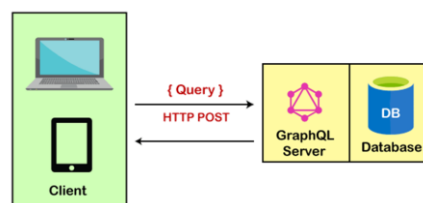
• **Graph Query Language (GraphQL)**

- GraphQL je povpraševalni jezik za API-je in izvajalno okolje, ki omogoča pridobivanje točno določenih podatkov.
- Omogoča optimizacijo prenosa podatkov, lažjo evolucijo.
- Razvilo ga je podjetje Facebook leta 2012 za izboljšanje fleksibilnosti in zmogljivosti API-jev
- Javno je bil objavljen v obliki odprtokodnega projekta leta **2015**.

45

GraphQL

- Jezik za definicijo vmesnika: **GraphQL Schema Definition Language** (SDL) omogoča strogo definicijo tipov in operacij (poizvedbe, mutacije, naročnine).
- Vsebina sporočila (payload): **Običajno JSON**, vendar ni omejeno na določen format.
- Prenosni protokol: **Primarno HTTP**, lahko pa tudi WebSocket za naročnine (subscriptions) ali druge protokole.
- Prva izdaja: **2015** (odprtokodno).



Vir: <https://www.javatpoint.com/graphql-architecture>

46

GraphQL - primer definicije vmesnika

```
# Definicija glavne sheme
schema {
  query: Query
  mutation: Mutation
}

# Tipi poizvedb
type Query {
  # Pridobi vse prometne senzorje
  getTrafficSensors: [TrafficSensor]

  # Pridobi podatke o prometnem toku
  getTrafficFlow(
    trafficId: String!,
    dateFrom: String!,
    dateTo: String!
  ): TrafficFlow
}

# Tipi mutacij
type Mutation {
  # Aktiviraj prometni senzor
  activateTrafficSensor(
    sensorName: String!,
    sensorType: String!,
    longitude: String!,
    latitude: String!
  ): TrafficSensorActivationResponse
}

# Definicija entitet
type TrafficSensor {
  sensorId: String!
  sensorName: String
  sensorType: String
  longitude: String
  latitude: String
  activationStatus: String
  activationDate: String
}

type TrafficFlow {
  trafficId: String!
  dateFrom: String
  dateTo: String
  trafficFlowData: String
}

# Odgovor na aktivacijo senzorja
type TrafficSensorActivationResponse {
  sensorId: String
  activationStatus: String
  activationDate: String
}
```

47

Glavni elementi GraphQL SDL

- **schema**
 - Določa glavne tipe operacij, ki jih API podpira, npr. query (poizvedbe) in mutation (mutacije za spreminjanje podatkov).
- **type**
 - Definira entitete (npr. TrafficSensor, TrafficFlow), ki jih API izpostavlja.
 - Vsak type vsebuje polja in njihove tipe podatkov (npr. String, Int, Boolean, ali druge tipe, ki jih definirate).
- **Query**
 - Določa vse poizvedbe, ki jih API podpira.
 - Polja znotraj Query ustrezajo metodam za pridobivanje podatkov.
- **Mutation**
 - Določa operacije, ki spreminjajo podatke, npr. dodajanje, brisanje ali posodabljanje entitet.
 - Polja znotraj Mutation ustrezajo metodam za manipulacijo podatkov.
- **Non-nullable tipi**
 - Tipi, kot je String!, označujejo, da je polje obvezno (ne sme biti null).

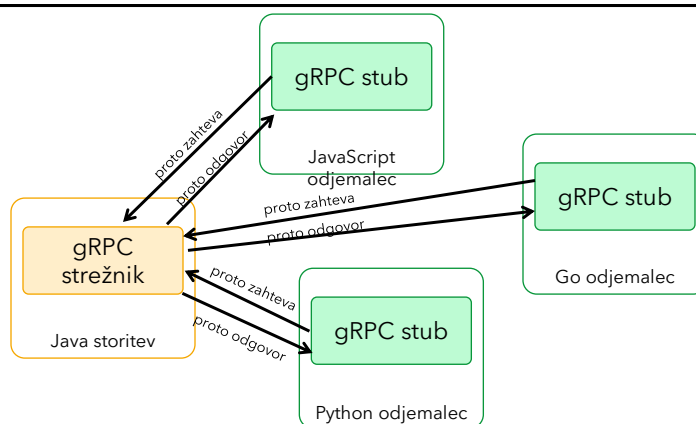
48

gRPC

- Sodobno ogrodje za razvoj rešitev tipa RPC
- Razvito iz strani podjetje Google kot odprtokodni projekt.
 - Omogoča visoko zmogljivo in transparentno komunikacijo med odjemalci in strežniki v različnih programskih jezikih in okoljih.
- Cilj: Poenostaviti razvoj povezanih sistemov z zagotavljanjem
 - učinkovitih komunikacijskih mehanizmov,
 - podpore za pretakanje podatkov in
 - dvosmerno komunikacijo.

49

gRPC



- Jezik za definicijo vmesnika: **Protocol Buffers** (Protobuf), ki omogočajo strogo tipizacijo in generiranje kode za odjemalce in strežnike v več programskih jezikih.
- Vsebina sporočila (payload): **Privzeto serializirani Protocol Buffers**, z možnostjo uporabe drugih formatov, kot je JSON.
- Prenosni protokol: **HTTP/2**, ki omogoča visoko zmogljivost, nizko latenco, dvosmerno komunikacijo in pretakanje.
- Prva izdaja: Odprtokodno objavljen leta **2015**, temelji pa na Googlovem internem sistemu Stubby.

50

```

syntax = "proto3";
package trafficService;
// Definicija gRPC storitve
service TrafficService {
    // Metoda za aktivacijo prometnega senzorja
    rpc TrafficSensorActivation (TrafficSensorActivationRequest) returns (TrafficSensorActivationResponse);
    // Metoda za pridobitev podatkov o prometnem toku
    rpc TrafficFlow (TrafficFlowRequest) returns (TrafficFlowResponse);
}
// Definicija sporočil za prometne senzorje
message TrafficSensorActivationRequest {
    string sensor_name = 1; // Ime senzorja
    string sensor_type = 2; // Vrsta senzorja
    string longitude = 3; // Geografska dolžina
    string latitude = 4; // Geografska širina
}
message TrafficSensorActivationResponse {
    string sensor_id = 1; // ID senzorja
    string activation_status = 2; // Status aktivacije
    string activation_date = 3; // Datum aktivacije
}
// Definicija sporočil za prometni tok
message TrafficFlowRequest {
    string traffic_id = 1; // ID prometnega toka
    string date_from = 2; // Začetni datum
    string date_to = 3; // Končni datum
}
message TrafficFlowResponse {
    string response = 1; // Odgovor s podatki o prometnem toku
}

```

GraphQL - primer definicije vmesnika

51

Glavni elementi proto

- **syntax = "proto3";**
 - Uporablja najnovejšo različico Protocol Buffers (proto3).
- **package**
 - Logična skupina, ki omogoča organizacijo sporočil in storitev (v tem primeru trafficService).
- **service**
 - Definicija gRPC storitve (TrafficService).
 - Vsaka metoda je definirana kot RPC, ki sprejema zahtevo in vrača odgovor.
- **message**
 - Struktura podatkov, ki se uporablja za vhodne in izhodne parametre RPC metod.
 - Polja v sporočilih so označena s številkami (npr. 1, 2), ki se uporabljajo za učinkovito serializacijo.
- **rpc**
 - Definira oddaljene metode (npr. TrafficSensorActivation, TrafficFlow).
 - Vsaka metoda določa vhodno (TrafficSensorActivationRequest) in izhodno (TrafficSensorActivationResponse) sporočilo.

52

RSocket

- RSocket je binarni aplikacijski protokol, ki ga je razvilo podjetje Netflix
 - Potem so pri razvoju sodelovali še Facebook, Netifi, Pivotal in drugi.
- Protokol je zasnovan za učinkovito prenašanje podatkovnih tokov med aplikacijami z uporabo različnih transportnih protokolov.
- Cilj je zamenjati HTTP, ki je pogosto neučinkovit pri scenarijih z visoko prepustnostjo, nizko latenco in dvosmerno komunikacijo, na primer pri mikrostoritvah.
- 2-smerni, na sporočilih temelječ binarni protokol, ki omogoča učinkovit prenos podatkov in ločevanje metapodatkov od vsebine.

53

RSocket

- Jezik za definicijo vmesnika: **odvisno od implementacije.**
 - **Na primer Rsocket RPC uporablja rešitev protocol buffer v3 od podjetja Google**
- Vsebina sporočila (payload):
 - RSocket omogoča ločevanje metapodatkov od dejanskih podatkov, kar omogoča prilagodljivost in optimizacijo komunikacije glede na zahteve aplikacije.
- Prenosni protokol:
 - RSocket deluje na več transportnih protokolih, vključno z TCP, WebSockets, Aeron in HTTP/2, kar omogoča fleksibilnost glede na omrežne potrebe.
- Prva izdaja:
 - Razvoj se je začel pri Netflixu okoli leta **2015**, protokol pa je bil uradno objavljen in odprtokodno izdan nekaj let kasneje.

54

Ključne funkcionalnosti RSocket

- Reaktivni model
 - Podpora za asinhrono operacije in pretok podatkov (streams).
- Nizka latenca
 - Optimizirano za visoko zmogljivost in nizko zakasnitev.
- Komunikacijski vzorci
 - Request/Response,
 - Fire-and-Forget,
 - Streaming (enosmerni ali dvosmerni).
- Podpora več protokolov
 - TCP, WebSocket, Aeron, itd.

55

Tehn./Protokol	Namen	Filozofija	Jezik za definicijo	Vsebina sporočila	Prenosni protokol	Prva izdaja	Posebnosti
CORBA	Porazdeljeni objekti	Objektni RPC	IDL	Binarni (IIOP)	TCP/IP	1991	Podpora za več jezikov, platformna neodvisnost
SOAP	Porazdeljene aplikacije	RPC na XML	WSDL	XML	HTTP, TCP	1999	Stroga standardizacija, visoka interoperabilnost
EJB	Poslovna logika	Komponent no orient.	Java Remote Interface	Serializirani objekti, JSON	Java RMI, HTTP	1998	Podpora za transakcije, varnost, zanesljivost
XML-RPC	Enostaven RPC	RPC na HTTP	Ni zahtevan	XML	HTTP	1998	Enostavnost, predhodnik SOAP
REST	Poenostavitev dostopa do virov	RESTful API	OpenAPI (opcijsko)	JSON, XML	HTTP	2000	Enostavnost, nizka sklopljenost, razširljivost
OData	Standardiziran dostop do podatkov	REST na steroidih	CSDL (JSON, XML)	JSON, XML	HTTP	2012	Podpora za poizvedbe, bogati metapodatki
gRPC	Visoko zmogljive RPC rešitve	RPC na HTTP/2	Protocol Buffers (Protobuf)	Binarni, JSON (opcijsko)	HTTP/2	2015	Hitrost, dvosmerno pretakanje, podpora za več jezikov
RSocket	Pretakanje podatkov	Dvosmerni protokol	Protocol Buffers (opcijsko)	Binarni	TCP, WebSockets, HTTP/2	2015	Optimiziran za pretakanje podatkov in minimalno porabo virov
GraphQL	Fleksibilen dostop do podatkov	Poizvedbe nad podatki	SDL	JSON	HTTP, WebSockets	2015	Natančne poizvedbe, introspekcija, en endpoint

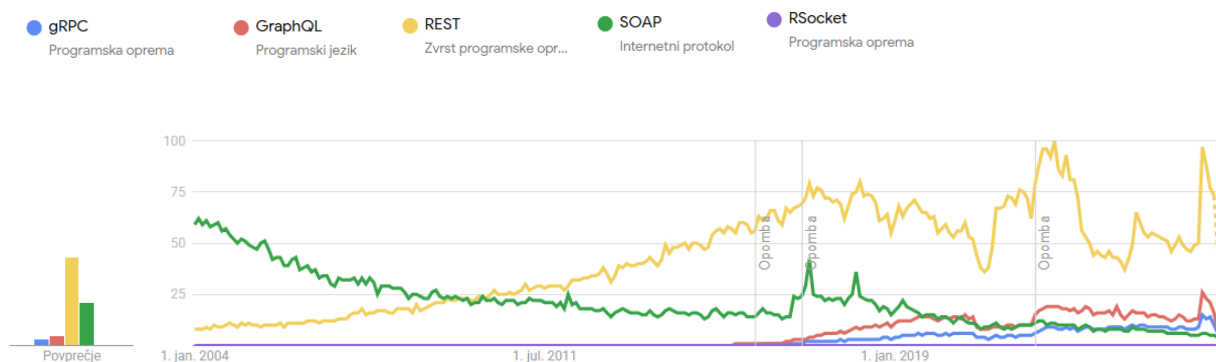
56

Principi SOA?

Tehn./ Protokol	Standardizacij a pogodb	Šibka sklopljenost	Abstrakcija	Ponovna uporaba	Avtonomnost	Brez stanja	Odkrivanje	Kompozicija
SOAP	✓	✓	✓	✓	✓	✓	✓	✓
REST	⚠	✓	✓	✓	✓	✓	⚠	⚠
gRPC	✓	✓	✓	✓	✓	✓	⚠	✓
RSocket	⚠	✓	✓	✓	✓	✓	✗	✓
CORBA	✓	⚠	✓	✓	✓	⚠	✓	✓
XML-RPC	✗	✓	✓	⚠	⚠	✓	✗	⚠
OData	✓	✓	✓	✓	✓	✓	✓	⚠
GraphQL	⚠	✓	✓	✓	✓	✓	✗	⚠
EJB	✓	⚠	✓	✓	✓	✗	✗	✓

57

APIji in tehnologije - zanimanje skozi čas



58